DLAA – Complete Class Code Reference

All Code from Jupyter Notebooks (Exact Copy)

Open Book Exam Reference

Contents

1 1-1: Transformers, What Can They Do (HuggingFace Pipelines) 3
1.1 Install . . . 3
1.2 Sentiment Analysis Pipeline . . . 3
1.3 Batch Sentiment Analysis . . . 3
1.4 Zero-Shot Classification . . . 3
1.5 Text Generation . . . 3
1.6 Fill-Mask . . . 3
1.7 Named Entity Recognition (NER) . . . 3
1.8 Question Answering . . . 3
1.9 Summarization . . . 4
1.10 Translation . . . 4

2 1-2: Basic Self-Attention (Step-by-Step Exercise) 5
2.1 Import . . . 5
2.2 Step 1: Prepare Inputs . . . 5
2.3 Step 2: Initialise Weights (Query, Key, Value) . . . 5
2.4 Step 3: Derive Key, Query and Value . . . 5
2.5 Step 4: Calculate Attention Scores . . . 6
2.6 Step 5: Calculate Softmax . . . 6
2.7 Step 6: Multiply Scores with Values . . . 6
2.8 Step 7: Sum Weighted Values to Get Output . . . 6

3 1-2 / 1B-1: Visualize Self-Attention with BertViz 7
3.1 Install . . . 7
3.2 Imports . . . 7
3.3 BertViz Attention Visualization (Updated Version) . . . 7
3.4 Head View and Model View . . . 7

4 1-3: BERT for Spam Email Classification 8
4.1 Imports and Setup . . . 8
4.2 Load Dataset . . . 8
4.3 Train/Val/Test Split . . . 8
4.4 Load BERT Model and Tokenizer . . . 8
4.5 Tokenization . . . 8
4.6 Convert to Tensors . . . 9
4.7 Freeze BERT Parameters . . . 9
4.8 Define Model Architecture . . . 9
4.9 Create DataLoaders . . . 10
4.10 Optimizer and Loss . . . 10
4.11 Training Function . . . 11
4.12 Evaluation Function . . . 11
4.13 Training Loop . . . 12
4.14 Load and Test . . . 13

5 1B-2: Audio Spectrogram Transformer (AST) 14
5.1 Install and Setup . . . 14
5.2 AST Model with Attention Visualization . . . 14
5.3 Load Pretrained Model . . . 15
5.4 Predict and Visualize Attention . . . 15
5.5 Visualize Mean Attention Maps . . . 15

6 1B-2: Text Metaphor Classification (BERT/RoBERTa) 17
6.1 Imports . . . 17
6.2 Config . . . 17
6.3 Load Data and Split . . . 17
6.4 Custom Dataset . . . 17
6.5 Tokenizer and DataLoader . . . 18
6.6 Model Definition . . . 18
6.7 Training Loop . . . 18
6.8 Test with Precision-Recall Curve . . . 19

7 1C-1: Vision Transformer (ViT) Fine-Tuning on CIFAR-10 (Example) 21
7.1 Load Dataset . . . 21
7.2 Label Mapping . . . 21
7.3 Feature Extractor and Transforms . . . 21
7.4 DataLoader and Collate . . . 22
7.5 Load ViT Model . . . 22
7.6 Training Arguments and Trainer . . . 22
7.7 Evaluation and Confusion Matrix . . . 22

8 1C-1a: ViT Exercise – Age Classification with Custom ViTLayer 24
8.1 Dataset Loading . . . 24
8.2 Custom ViTLayer (Residual Connections) . . . 24
8.3 Load Model with Custom Layer (mock.patch) . . . 24
8.4 Training with HF Trainer . . . 25
8.5 Evaluation with Confusion Matrix and Classification Report . . . 25

9 1C-2: Swin Transformer Fine-Tuning (Butterfly Classification) 26
9.1 Imports . . . 26
9.2 Data Loading Functions . . . 26
9.3 Model Setup (Freeze + Custom Head) . . . 26
9.4 Training Function . . . 27
9.5 Testing . . . 27

10 1D-1: Detection Transformer (DETR) Fine-Tuning 29
10.1 COCO Dataset Class . . . 29
10.2 Feature Extractor and Datasets . . . 29
10.3 DataLoader with Padding Collate . . . 29
10.4 DETR Lightning Module . . . 30
10.5 Train and Evaluate . . . 31
10.6 COCO Evaluation . . . 31
10.7 Visualization Helpers . . . 31

11 1D-2: DAB-DETR Inference 33
11.1 Load Model . . . 33
11.2 Fetch COCO Images . . . 33
11.3 Detection and Visualization . . . 33
11.4 Statistics . . . 34

12 1E-1: ViT Self-Attention Exercise (DINO Attention Visualization) 35
12.1 ViTSelfAttentionExercise Class . . . 35
12.2 Test Script (Weight Copy and Verification) . . . 35
12.3 DINO Attention Visualization (Notebook) . . . 36

1 1-1: Transformers, What Can They Do (HuggingFace Pipelines)

Keywords: pipeline, sentiment-analysis, zero-shot-classification, text-generation, fill-mask, ner, question-answering, summarization, translation

1.1 Install

```
!pip install datasets evaluate transformers[sentencepiece]
```

1.2 Sentiment Analysis Pipeline

```
from transformers import pipeline

classifier = pipeline("sentiment-analysis")
classifier("I've been waiting for a HuggingFace course my whole life.")
```

1.3 Batch Sentiment Analysis

```
classifier(
    ["I've been waiting for a HuggingFace course my whole life.", "I hate this so much!"]
)
```

1.4 Zero-Shot Classification

```
from transformers import pipeline

classifier = pipeline("zero-shot-classification")
classifier(
    "This is a course about the Transformers library",
    candidate_labels=["education", "politics", "business"],
)
```

1.5 Text Generation

```
from transformers import pipeline

generator = pipeline("text-generation")
generator("In this course, we will teach you how to")
```

1.6 Fill-Mask

```
from transformers import pipeline

unmasker = pipeline("fill-mask")
unmasker("This course will teach you all about <mask> models.", top_k=2)
```

1.7 Named Entity Recognition (NER)

```
from transformers import pipeline

ner = pipeline("ner", grouped_entities=True)
ner("My name is Sylvain and I work at Hugging Face in Brooklyn.")
```

1.8 Question Answering

```
from transformers import pipeline

question_answerer = pipeline("question-answering")
question_answerer(
    question="Where do I work?",
    context="My name is Sylvain and I work at Hugging Face in Brooklyn",
)
```

1.9 Summarization

```python
from transformers import pipeline

summarizer = pipeline("summarization")
summarizer(
    """
    America has changed dramatically during recent years. Not only has the number of
    graduates in traditional engineering disciplines such as mechanical, civil,
    electrical, chemical, and aeronautical engineering declined, but in most of
    the premier American universities engineering curricula now concentrate on
    and encourage largely the study of engineering science. As a result, there
    are declining offerings in engineering subjects dealing with infrastructure,
    the environment, and related issues, and greater concentration on high
    technology subjects, largely supporting increasingly complex scientific
    developments. While the latter is important, it should not be at the expense
    of more traditional engineering.

    Rapidly developing economies such as China and India, as well as other
    industrial countries in Europe and Asia, continue to encourage and advance
    the teaching of engineering. Both China and India, respectively, graduate
    six and eight times as many traditional engineers as does the United States.
    Other industrial countries at minimum maintain their output, while America
    suffers an increasingly serious decline in the number of engineering graduates
    and a lack of well-educated engineers.
"""
)
```

1.10 Translation

```python
from transformers import pipeline

translator = pipeline("translation", model="Helsinki-NLP/opus-mt-fr-en")
translator("Ce cours est produit par Hugging Face.")
```

2 1-2: Basic Self-Attention (Step-by-Step Exercise)

Keywords: `self-attention, query, key, value, dot-product, softmax, weighted values, torch.matmul`

2.1 Import

```
import torch
```

2.2 Step 1: Prepare Inputs

```
x = [
  [], #modify the input 1
  [], #modify the input 2
  []  #modify the input 3
 ]
x = torch.tensor(x, dtype=torch.float32)
x
```

2.3 Step 2: Initialise Weights (Query, Key, Value)

```
# Define the weights for the query(w_query), key(w_key) and value(w_value)
w_query = [
  [1, 0, 1],
  [1, 0, 0],
  [0, 0, 1],
  [0, 1, 1]
]
w_key = [
  [0, 0, 1],
  [1, 1, 0],
  [0, 1, 0],
  [1, 1, 0]
]
w_value = [
  [0, 2, 0],
  [0, 3, 0],
  [1, 0, 3],
  [1, 1, 0]
]
w_query = torch.tensor(w_query, dtype=torch.float32)
w_key = torch.tensor(w_key, dtype=torch.float32)
w_value = torch.tensor(w_value, dtype=torch.float32)

print("Weights for query: \n", w_query)
print("Weights for key: \n", w_key)
print("Weights for value: \n", w_value)
```

2.4 Step 3: Derive Key, Query and Value

```
querys = x @ w_query
keys = x @ w_key
values = x @ w_value


print("Querys: \n", querys)
# tensor([[1., 0., 2.],
#         [2., 2., 2.],
#         [2., 1., 3.]])
print("Keys: \n", keys)
# tensor([[0., 1., 1.],
#         [4., 4., 0.],
#         [2., 3., 1.]])
print("Values: \n", values)
# tensor([[1., 2., 3.],
#         [2., 8., 0.],
#         [2., 6., 3.]])
```

2.5 Step 4: Calculate Attention Scores

```
attn_scores = querys @ keys.T
print(attn_scores)

# tensor([[ 2.,  4.,  4.],  # attention scores from Query 1
#         [ 4., 16., 12.],  # attention scores from Query 2
#         [ 4., 12., 10.]]) # attention scores from Query 3
```

2.6 Step 5: Calculate Softmax

```
from torch.nn.functional import softmax

attn_scores_softmax = softmax(attn_scores, dim=-1)
print(attn_scores_softmax)

# For readability, approximate the above as follows
attn_scores_softmax = [
  [0.0, 0.5, 0.5],
  [0.0, 1.0, 0.0],
  [0.0, 0.9, 0.1]
]
attn_scores_softmax = torch.tensor(attn_scores_softmax)
print(attn_scores_softmax)
```

2.7 Step 6: Multiply Scores with Values

```
weighted_values = values[:,None] * attn_scores_softmax.T[:,:,None]
print(weighted_values)
```

2.8 Step 7: Sum Weighted Values to Get Output

```
outputs = weighted_values.sum(dim=0)
print(outputs)

# tensor([[2.0000, 7.0000, 1.5000],  # Output 1
#         [2.0000, 8.0000, 0.0000],  # Output 2
#         [2.0000, 7.8000, 0.3000]]) # Output 3
```

3 1-2 / 1B-1: Visualize Self-Attention with BertViz

Keywords: bertviz, head_view, model_view, BERT, attention visualization, tokenizer, encode_plus, token_type

3.1 Install

```
!pip install transformers
!pip install bertviz
```

3.2 Imports

```
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
import transformers
from transformers import AutoModel, BertTokenizerFast

# specify GPU
device = torch.device("cuda")
```

3.3 BertViz Attention Visualization (Updated Version)

```
from bertviz import head_view, model_view
from transformers import AutoTokenizer, AutoModel, AutoConfig, utils
utils.logging.set_verbosity_error()  # Suppress standard warnings

# Create a configuration object and set output_attentions to True
config = AutoConfig.from_pretrained('bert-base-uncased', output_attentions=True)

# Initialize the model with the modified configuration
bert = AutoModel.from_pretrained('bert-base-uncased', config=config)

# Load the BERT tokenizer
tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')
sentence_a = "The rabbit quickly hopped"
sentence_b = "The turtle slowly crawled"

# Directly call the tokenizer for encoding
inputs = tokenizer(sentence_a, sentence_b, return_tensors='pt')

input_ids = inputs['input_ids']
token_type_ids = inputs['token_type_ids'] # token type id is 0 for Sentence A and 1 for Sentence B
```

3.4 Head View and Model View

```
attention = bert(input_ids, token_type_ids=token_type_ids,output_attentions=True)[-1]

sentence_b_start = token_type_ids[0].tolist().index(1) # Sentence B starts at first index of token
    type id 1
token_ids = input_ids[0].tolist() # Batch index 0
tokens = tokenizer.convert_ids_to_tokens(token_ids)
head_view(attention, tokens, sentence_b_start)
```

```
model_view(attention, tokens, include_layers=[0,1,2,3,4,5,6,7,8,9,10,11])
```

4 1-3: BERT for Spam Email Classification

Keywords: BERT, fine-tune, classification, tokenizer, DataLoader, NLLLoss, train loop, evaluate, class weights, freeze parameters

4.1 Imports and Setup

```
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
import transformers
from transformers import AutoModel, BertTokenizerFast

# specify GPU
device = torch.device("cuda")
```

4.2 Load Dataset

```
df = pd.read_csv("spamdata_v2.csv")
df.head()
```

```
df.shape
```

```
# check class distribution
df['label'].value_counts(normalize = True)
```

4.3 Train/Val/Test Split

```
train_text, temp_text, train_labels, temp_labels = train_test_split(df['text'], df['label'],
                                      random_state=2018,
                                      test_size=0.3,
                                      stratify=df['label'])

# we will use temp_text and temp_labels to create validation and test set
val_text, test_text, val_labels, test_labels = train_test_split(temp_text, temp_labels,
                                    random_state=2018,
                                    test_size=0.5,
                                    stratify=temp_labels)
```

4.4 Load BERT Model and Tokenizer

```
# import BERT-base pretrained model
bert = AutoModel.from_pretrained('bert-base-uncased')

# Load the BERT tokenizer
tokenizer = BertTokenizerFast.from_pretrained('bert-base-uncased')
```

4.5 Tokenization

```
# sample data
text = ["this is a bert model tutorial", "we will fine-tune a bert model"]

# encode text
sent_id = tokenizer.batch_encode_plus(text, padding=True, return_token_type_ids=False)
```

```
# get length of all the messages in the train set
seq_len = [len(i.split()) for i in train_text]

pd.Series(seq_len).hist(bins = 30)
```

```
max_seq_len = 25
```

```python
# tokenize and encode sequences in the training set
tokens_train = tokenizer.batch_encode_plus(
    train_text.tolist(),
    max_length = max_seq_len,
    padding='max_length',
    truncation=True,
    return_token_type_ids=False
)

# tokenize and encode sequences in the validation set
tokens_val = tokenizer.batch_encode_plus(
    val_text.tolist(),
    max_length = max_seq_len,
    padding='max_length',
    truncation=True,
    return_token_type_ids=False
)

# tokenize and encode sequences in the test set
tokens_test = tokenizer.batch_encode_plus(
    test_text.tolist(),
    max_length = max_seq_len,
    padding='max_length',
    truncation=True,
    return_token_type_ids=False
)
```

4.6 Convert to Tensors

```python
# for train set
train_seq = torch.tensor(tokens_train['input_ids'])
train_mask = torch.tensor(tokens_train['attention_mask'])
train_y = torch.tensor(train_labels.tolist())

# for validation set
val_seq = torch.tensor(tokens_val['input_ids'])
val_mask = torch.tensor(tokens_val['attention_mask'])
val_y = torch.tensor(val_labels.tolist())

# for test set
test_seq = torch.tensor(tokens_test['input_ids'])
test_mask = torch.tensor(tokens_test['attention_mask'])
test_y = torch.tensor(test_labels.tolist())
```

4.7 Freeze BERT Parameters

```python
# freeze all the parameters
for param in bert.parameters():
    param.requires_grad = False
```

4.8 Define Model Architecture

```python
class BERT_Arch(nn.Module):

    def __init__(self, bert):

      super(BERT_Arch, self).__init__()

      self.bert = bert

      # dropout layer
      self.dropout = nn.Dropout(0.1)

      # relu activation function
      self.relu =  nn.ReLU()

      # dense layer 1
      self.fc1 = nn.Linear(768,512)

      # dense layer 2 (Output layer)
      self.fc2 = nn.Linear(512,2)

```

```
      #softmax activation function
      self.softmax = nn.LogSoftmax(dim=1)

    #define the forward pass
    def forward(self, sent_id, mask):

      #pass the inputs to the model
      _, cls_hs = self.bert(sent_id, attention_mask=mask, return_dict=False)
      x = self.fc1(cls_hs)

      x = self.relu(x)

      x = self.dropout(x)

      # output layer
      x = self.fc2(x)

      # apply softmax activation
      x = self.softmax(x)

      return x
```

4.9 Create DataLoaders

```
# pass the pre-trained BERT to our define architecture
model = BERT_Arch(bert)

# push the model to GPU
model = model.to(device)
```

```
from torch.utils.data import TensorDataset, DataLoader, RandomSampler, SequentialSampler

#define a batch size
batch_size = 32

# wrap tensors
train_data = TensorDataset(train_seq, train_mask, train_y)

# sampler for sampling the data during training
train_sampler = RandomSampler(train_data)

# dataLoader for train set
train_dataloader = DataLoader(train_data, sampler=train_sampler, batch_size=batch_size)

# wrap tensors
val_data = TensorDataset(val_seq, val_mask, val_y)

# sampler for sampling the data during training
val_sampler = SequentialSampler(val_data)

# dataLoader for validation set
val_dataloader = DataLoader(val_data, sampler = val_sampler, batch_size=batch_size)
```

4.10 Optimizer and Loss

```
# optimizer from hugging face transformers
from transformers import AdamW

# define the optimizer
optimizer = AdamW(model.parameters(), lr = 1e-3)
```

```
from sklearn.utils.class_weight import compute_class_weight

#compute the class weights
class_wts = compute_class_weight( class_weight = "balanced", classes = np.unique(train_labels),y =
    train_labels)

print(class_wts)
```

```
# convert class weights to tensor
weights= torch.tensor(class_wts,dtype=torch.float)
weights = weights.to(device)

```

```
# loss function
cross_entropy  = nn.NLLLoss(weight=weights)

# number of training epochs
epochs = 10
```

4.11 Training Function

```
# function to train the model
def train():

  model.train()

  total_loss, total_accuracy = 0, 0

  # empty list to save model predictions
  total_preds=[]

  # iterate over batches
  for step,batch in enumerate(train_dataloader):

    # progress update after every 50 batches.
    if step % 50 == 0 and not step == 0:
      print('  Batch {:>5,}  of  {:>5,}.'.format(step, len(train_dataloader)))

    # push the batch to gpu
    batch = [r.to(device) for r in batch]

    sent_id, mask, labels = batch

    # clear previously calculated gradients
    model.zero_grad()

    # get model predictions for the current batch
    preds = model(sent_id, mask)

    # compute the loss between actual and predicted values
    loss = cross_entropy(preds, labels)

    # add on to the total loss
    total_loss = total_loss + loss.item()

    # backward pass to calculate the gradients
    loss.backward()

    # clip the the gradients to 1.0. It helps in preventing the exploding gradient problem
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)

    # update parameters
    optimizer.step()

    # model predictions are stored on GPU. So, push it to CPU
    preds=preds.detach().cpu().numpy()

    # append the model predictions
    total_preds.append(preds)

  # compute the training loss of the epoch
  avg_loss = total_loss / len(train_dataloader)

  # predictions are in the form of (no. of batches, size of batch, no. of classes).
  # reshape the predictions in form of (number of samples, no. of classes)
  total_preds  = np.concatenate(total_preds, axis=0)

  #returns the loss and predictions
  return avg_loss, total_preds
```

4.12 Evaluation Function

```
# function for evaluating the model
def evaluate():

  print("\nEvaluating...")

  # deactivate dropout layers
```

```
  model.eval()

  total_loss, total_accuracy = 0, 0

  # empty list to save the model predictions
  total_preds = []

  # iterate over batches
  for step,batch in enumerate(val_dataloader):

    # Progress update every 50 batches.
    if step % 50 == 0 and not step == 0:

      # Calculate elapsed time in minutes.
      elapsed = format_time(time.time() - t0)

      # Report progress.
      print('  Batch {:>5,}  of  {:>5,}.'.format(step, len(val_dataloader)))

    # push the batch to gpu
    batch = [t.to(device) for t in batch]

    sent_id, mask, labels = batch

    # deactivate autograd
    with torch.no_grad():

      # model predictions
      preds = model(sent_id, mask)

      # compute the validation loss between actual and predicted values
      loss = cross_entropy(preds,labels)

      total_loss = total_loss + loss.item()

      preds = preds.detach().cpu().numpy()

      total_preds.append(preds)

  # compute the validation loss of the epoch
  avg_loss = total_loss / len(val_dataloader)

  # reshape the predictions in form of (number of samples, no. of classes)
  total_preds  = np.concatenate(total_preds, axis=0)

  return avg_loss, total_preds
```

4.13 Training Loop

```
# set initial loss to infinite
best_valid_loss = float('inf')

# empty lists to store training and validation loss of each epoch
train_losses=[]
valid_losses=[]

#for each epoch
for epoch in range(epochs):

    print('\n Epoch {:} / {:}'.format(epoch + 1, epochs))

    #train model
    train_loss, _ = train()

    #evaluate model
    valid_loss, _ = evaluate()

    #save the best model
    if valid_loss < best_valid_loss:
        best_valid_loss = valid_loss
        torch.save(model.state_dict(), 'saved_weights.pt')

    # append training and validation loss
    train_losses.append(train_loss)
    valid_losses.append(valid_loss)

    print(f'\nTraining Loss: {train_loss:.3f}')
```

```
    print(f'Validation Loss: {valid_loss:.3f}')
```

4.14 Load and Test

```
#load weights of best model
path = 'saved_weights.pt'
model.load_state_dict(torch.load(path))
```

```
# get predictions for test data
with torch.no_grad():
  preds = model(test_seq.to(device), test_mask.to(device))
  preds = preds.detach().cpu().numpy()
```

```
# model's performance
preds = np.argmax(preds, axis = 1)
print(classification_report(test_y, preds))
```

```
# confusion matrix
pd.crosstab(test_y, preds)
```

5 1B-2: Audio Spectrogram Transformer (AST)

Keywords: AST, audio, spectrogram, torchaudio, fbank, mel_bins, attention map visualization, AudioSet

5.1 Install and Setup

```
import sys
if 'google.colab' in sys.modules:
    print('Running in Colab.')
    !git clone https://github.com/YuanGongND/ast
    sys.path.append('./ast')
%cd /content/ast/

! pip install timm==0.4.5
! pip install wget
import os, csv, argparse, wget
os.environ['TORCH_HOME'] = '/content/ast/pretrained_models'
if os.path.exists('/content/ast/pretrained_models') == False:
  os.mkdir('/content/ast/pretrained_models')
import torch, torchaudio, timm
import numpy as np
from torch.cuda.amp import autocast
import IPython
```

5.2 AST Model with Attention Visualization

```
from src.models import ASTModel

# Create a new class that inherits the original ASTModel class
class ASTModelVis(ASTModel):
    def get_att_map(self, block, x):
        qkv = block.attn.qkv
        num_heads = block.attn.num_heads
        scale = block.attn.scale
        B, N, C = x.shape
        qkv = qkv(x).reshape(B, N, 3, num_heads, C // num_heads).permute(2, 0, 3, 1, 4)
        q, k, v = qkv[0], qkv[1], qkv[2]
        attn = (q @ k.transpose(-2, -1)) * scale
        attn = attn.softmax(dim=-1)
        return attn

    def forward_visualization(self, x):
        # expect input x = (batch_size, time_frame_num, frequency_bins), e.g., (12, 1024, 128)
        x = x.unsqueeze(1)
        x = x.transpose(2, 3)

        B = x.shape[0]
        x = self.v.patch_embed(x)
        cls_tokens = self.v.cls_token.expand(B, -1, -1)
        dist_token = self.v.dist_token.expand(B, -1, -1)
        x = torch.cat((cls_tokens, dist_token, x), dim=1)
        x = x + self.v.pos_embed
        x = self.v.pos_drop(x)
        # save the attention map of each of 12 Transformer layer
        att_list = []
        for blk in self.v.blocks:
            cur_att = self.get_att_map(blk, x)
            att_list.append(cur_att)
            x = blk(x)
        return att_list

def make_features(wav_name, mel_bins, target_length=1024):
    waveform, sr = torchaudio.load(wav_name)
    assert sr == 16000, 'input audio sampling rate must be 16kHz'

    fbank = torchaudio.compliance.kaldi.fbank(
        waveform, htk_compat=True, sample_frequency=sr, use_energy=False,
        window_type='hanning', num_mel_bins=mel_bins, dither=0.0, frame_shift=10)

    n_frames = fbank.shape[0]

    p = target_length - n_frames
    if p > 0:
        m = torch.nn.ZeroPad2d((0, 0, 0, p))
        fbank = m(fbank)
```

```
    elif p < 0:
        fbank = fbank[0:target_length, :]

    fbank = (fbank - (-4.2677393)) / (4.5689974 * 2)
    return fbank


def load_label(label_csv):
    with open(label_csv, 'r') as f:
        reader = csv.reader(f, delimiter=',')
        lines = list(reader)
    labels = []
    ids = []
    for i1 in range(1, len(lines)):
        id = lines[i1][1]
        label = lines[i1][2]
        ids.append(id)
        labels.append(label)
    return labels
```

5.3 Load Pretrained Model

```
audioset_mdl_url = 'https://www.dropbox.com/s/cv4knew8mvbrnvq/audioset_0.4593.pth?dl=1'
if os.path.exists('/content/ast/pretrained_models/audio_mdl.pth') == False:
  wget.download(audioset_mdl_url, out='/content/ast/pretrained_models/audio_mdl.pth')

input_tdim = 1024
checkpoint_path = '/content/ast/pretrained_models/audio_mdl.pth'
ast_mdl = ASTModelVis(label_dim=527, input_tdim=input_tdim, imagenet_pretrain=False,
    audioset_pretrain=False)
print(f'[*INFO] load checkpoint: {checkpoint_path}')
checkpoint = torch.load(checkpoint_path, map_location='cuda')
audio_model = torch.nn.DataParallel(ast_mdl, device_ids=[0])
audio_model.load_state_dict(checkpoint)
audio_model = audio_model.to(torch.device("cuda:0"))
audio_model.eval()

label_csv = '/content/ast/egs/audioset/data/class_labels_indices.csv'
labels = load_label(label_csv)
```

5.4 Predict and Visualize Attention

```
sample_audio_path = 'https://www.dropbox.com/s/vddohcnb9ane9ag/LDoXsip0BEQ_000177.flac?dl=1'

if os.path.exists('/content/ast/sample_audios') == False:
  os.mkdir('/content/ast/sample_audios')
if os.path.exists('/content/ast/sample_audios/sample_audio.flac') == True:
  os.remove('/content/ast/sample_audios/sample_audio.flac')
wget.download(sample_audio_path, '/content/ast/sample_audios/sample_audio.flac')
feats = make_features('/content/ast/sample_audios/sample_audio.flac', mel_bins=128)
feats_data = feats.expand(1, input_tdim, 128)
feats_data = feats_data.to(torch.device("cuda:0"))

with torch.no_grad():
  with autocast():
    output = audio_model.forward(feats_data)
    output = torch.sigmoid(output)
result_output = output.data.cpu().numpy()[0]
sorted_indexes = np.argsort(result_output)[::-1]

print('Predice results:')
for k in range(10):
    print('- {}: {:.4f}'.format(np.array(labels)[sorted_indexes[k]], result_output[sorted_indexes[k
    ]]))
```

5.5 Visualize Mean Attention Maps

```
from matplotlib import pyplot as plt
plt.imshow(feats_data[0].t().cpu(), origin='lower')
plt.title('Original Spectrogram')
plt.show()
plt.close()
with torch.no_grad():
```

```
  with autocast():
    att_list = audio_model.module.forward_visualization(feats_data)
for i in range(len(att_list)):
  att_list[i] = att_list[i].data.cpu().numpy()
  att_list[i] = np.mean(att_list[i][0], axis=0)
  att_list[i] = np.mean(att_list[i][0:2], axis=0)
  att_list[i] = att_list[i][2:].reshape(12, 101)
  plt.imshow(att_list[i], origin='lower')
  plt.title('Mean Attention Map of Layer #{:d}'.format(i))
  plt.show()
  plt.close()
```

6 1B-2: Text Metaphor Classification (BERT/RoBERTa)

Keywords: RoBERTa, BERT, text classification, Dataset, DataLoader, collate_fn, BCELoss, Sigmoid, training loop, validation

6.1 Imports

```
import os
import pandas
import torch
from torch import nn
from torch.utils.data import Dataset, DataLoader
from transformers import AutoTokenizer
from transformers import AutoModel
from pathlib import Path
from tqdm.notebook import tqdm
```

6.2 Config

```
batch_size = 8

text_max_length = 128
epochs = 3

validation_ratio = 0.1
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

log_per_step = 50

dataset_dir = Path("./dataset")
os.makedirs(dataset_dir) if not os.path.exists(dataset_dir) else ''

model_dir = Path("./model/bert_checkpoints")
os.makedirs(model_dir) if not os.path.exists(model_dir) else ''

print("Device:", device)
```

6.3 Load Data and Split

```
pd_data = pandas.read_csv("/content/dataset/train.csv")[['text', 'target']]
```

```
pd_validation_data = pd_data.sample(frac=validation_ratio)
pd_train_data = pd_data[~pd_data.index.isin(pd_validation_data.index)]
```

6.4 Custom Dataset

```
class MyDataset(Dataset):

    def __init__(self, mode='train'):
        super(MyDataset, self).__init__()
        self.mode = mode

        if mode == 'train':
            self.dataset = pd_train_data
        elif mode == 'validation':
            self.dataset = pd_validation_data
        elif mode == 'test':
            self.dataset = pandas.read_csv('./test.csv')[['text', 'id']]
        else:
            raise Exception("Unknown mode {}".format(mode))

    def __getitem__(self, index):
        data = self.dataset.iloc[index]
        source = data['text'].replace("#", "").replace("@", "")
        if self.mode == 'test':
            target = data['id']
        else:
            target = data['target']
        return source, target

```

```
    def __len__(self):
        return len(self.dataset)
```

6.5 Tokenizer and DataLoader

```
from transformers import RobertaForSequenceClassification, RobertaTokenizer
import torch

tokenizer = RobertaTokenizer.from_pretrained("roberta-large")
roberta_model = RobertaForSequenceClassification.from_pretrained("roberta-large")

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
roberta_model.to(device)
```

```
def collate_fn(batch):
    text, target = zip(*batch)
    text, target = list(text), list(target)

    src = tokenizer(text, padding='max_length', max_length=text_max_length, return_tensors='pt',
    truncation=True)

    return src, torch.LongTensor(target)
```

```
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True, collate_fn=collate_fn)
validation_loader = DataLoader(validation_dataset, batch_size=batch_size, shuffle=False, collate_fn=
    collate_fn)
```

6.6 Model Definition

```
class MyModel(nn.Module):

    def __init__(self):
        super(MyModel, self).__init__()

        self.bert = AutoModel.from_pretrained("roberta-large")

        self.predictor = nn.Sequential(
            nn.Linear(1024, 256),
            nn.ReLU(),
            nn.Linear(256, 1),
            nn.Sigmoid()
        )

    def forward(self, src):
        outputs = self.bert(**src).last_hidden_state[:, 0, :]
        return self.predictor(outputs)
```

6.7 Training Loop

```
criteria = nn.BCELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=3e-5)
```

```
def to_device(dict_tensors):
    result_tensors = {}
    for key, value in dict_tensors.items():
        result_tensors[key] = value.to(device)
    return result_tensors
```

```
def validate():
    model.eval()
    total_loss = 0.
    total_correct = 0
    for inputs, targets in validation_loader:
        inputs, targets = to_device(inputs), targets.to(device)
        outputs = model(inputs)
        loss = criteria(outputs.view(-1), targets.float())
        total_loss += float(loss)

        correct_num = (((outputs >= 0.5).float() * 1).flatten() == targets).sum()
        total_correct += correct_num

    return total_correct / len(validation_dataset), total_loss / len(validation_dataset)
```

```python
model.train()

if torch.cuda.is_available():
    torch.cuda.empty_cache()

total_loss = 0.
step = 0
best_accuracy = 0

for epoch in range(epochs):
    model.train()
    for i, (inputs, targets) in enumerate(train_loader):

        inputs, targets = to_device(inputs), targets.to(device)
        outputs = model(inputs)
        loss = criteria(outputs.view(-1), targets.float())
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()

        total_loss += float(loss)
        step += 1

        if step % log_per_step == 0:
            print("Epoch {}/{}, Step: {}/{}, total loss:{:.4f}".format(epoch+1, epochs, i, len(train_loader), total_loss))
            total_loss = 0

        del inputs, targets

    accuracy, validation_loss = validate()
    print("Epoch {}, accuracy: {:.4f}, validation loss: {:.4f}".format(epoch+1, accuracy, validation_loss))
    torch.save(model, model_dir / f"model_{epoch}.pt")

    if accuracy > best_accuracy:
        torch.save(model, model_dir / f"model_best.pt")
        best_accuracy = accuracy
```

6.8 Test with Precision-Recall Curve

```python
from sklearn.metrics import precision_recall_curve
import matplotlib.pyplot as plt

def test():
    model.eval()
    total_loss = 0.
    total_correct = 0
    TP, TN, FP, FN = 0, 0, 0, 0

    all_targets = []
    all_outputs = []

    for inputs, targets in test_loader:
        inputs, targets = to_device(inputs), targets.to(device)
        outputs = model(inputs)
        loss = criteria(outputs.view(-1), targets.float())
        total_loss += float(loss)

        predicted_labels = (outputs >= 0.5).float()
        predicted_labels = predicted_labels.cpu()
        targets = targets.cpu()
        TP += ((predicted_labels == 1) & (targets == 1)).sum().item()
        TN += ((predicted_labels == 0) & (targets == 0)).sum().item()
        FP += ((predicted_labels == 1) & (targets == 1)).sum().item()
        FN += ((predicted_labels == 0) & (targets == 0)).sum().item()

        all_targets.extend(targets.tolist())
        all_outputs.extend(outputs.tolist())

        total_correct += ((predicted_labels == targets.unsqueeze(1)).sum()).item()

    accuracy = total_correct / len(test_dataset)
    test_loss = total_loss / len(test_dataset)

    precision, recall, _ = precision_recall_curve(all_targets, all_outputs)

```

```
    plt.figure()
    plt.plot(recall, precision, marker='.')
    plt.xlabel('Recall')
    plt.ylabel('Precision')
    plt.title('Precision-Recall Curve')
    plt.grid(True)
    plt.show()

    return accuracy, test_loss, TP, TN, FP, FN
```

7 1C-1: Vision Transformer (ViT) Fine-Tuning on CIFAR-10 (Example)

Keywords: ViT, Vision Transformer, CIFAR-10, ViTImageProcessor, ViTForImageClassification, Trainer, transforms, fine-tune, confusion matrix

7.1 Load Dataset

```
from datasets import load_dataset

train_ds, test_ds = load_dataset('cifar10', split=['train[:5000]', 'test[:2000]'])
splits = train_ds.train_test_split(test_size=0.1)
train_ds = splits['train']
val_ds = splits['test']
```

7.2 Label Mapping

```
id2label = {id:label for id, label in enumerate(train_ds.features['label'].names)}
label2id = {label:id for id,label in id2label.items()}
id2label
```

7.3 Feature Extractor and Transforms

```
from transformers import ViTImageProcessor

feature_extractor = ViTImageProcessor.from_pretrained("google/vit-base-patch16-224-in21k")
```

```
from torchvision.transforms import (CenterCrop,
                    Compose,
                    Normalize,
                    RandomHorizontalFlip,
                    RandomResizedCrop,
                    Resize,
                    ToTensor)

normalize = Normalize(mean=feature_extractor.image_mean, std=feature_extractor.image_std)
size = (feature_extractor.size['height'], feature_extractor.size['width'])
_train_transforms = Compose(
        [
            RandomResizedCrop(size),
            RandomHorizontalFlip(),
            ToTensor(),
            normalize,
        ]
    )

_val_transforms = Compose(
        [
            Resize(size),
            CenterCrop(size),
            ToTensor(),
            normalize,
        ]
    )

def train_transforms(examples):
    examples['pixel_values'] = [_train_transforms(image.convert("RGB")) for image in examples['img']]
    return examples

def val_transforms(examples):
    examples['pixel_values'] = [_val_transforms(image.convert("RGB")) for image in examples['img']]
    return examples
```

```
train_ds.set_transform(train_transforms)
val_ds.set_transform(val_transforms)
test_ds.set_transform(val_transforms)
```

7.4 DataLoader and Collate

```
from torch.utils.data import DataLoader
import torch

def collate_fn(examples):
    pixel_values = torch.stack([example["pixel_values"] for example in examples])
    labels = torch.tensor([example["label"] for example in examples])
    return {"pixel_values": pixel_values, "labels": labels}

train_dataloader = DataLoader(train_ds, collate_fn=collate_fn, batch_size=4)
```

7.5 Load ViT Model

```
from transformers import ViTForImageClassification

model = ViTForImageClassification.from_pretrained('google/vit-base-patch16-224-in21k',
                            num_labels=10,
                            id2label=id2label,
                            label2id=label2id)
```

7.6 Training Arguments and Trainer

```
from transformers import TrainingArguments, Trainer

metric_name = "accuracy"

args = TrainingArguments(
    f"test-cifar-10",
    save_strategy="epoch",
    eval_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=10,
    per_device_eval_batch_size=4,
    num_train_epochs=5,
    weight_decay=0.01,
    load_best_model_at_end=True,
    metric_for_best_model=metric_name,
    logging_dir='logs',
    remove_unused_columns=False,
)
```

```
import evaluate
import numpy as np

metric = evaluate.load("accuracy")

def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    predictions = np.argmax(predictions, axis=1)
    return metric.compute(predictions=predictions, references=labels)
```

```
trainer = Trainer(
    model,
    args,
    train_dataset=train_ds,
    eval_dataset=val_ds,
    data_collator=collate_fn,
    compute_metrics=compute_metrics
)
```

```
trainer.train()
```

7.7 Evaluation and Confusion Matrix

```
outputs = trainer.predict(test_ds)
print(outputs.metrics)
```

```python
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

y_true = outputs.label_ids
y_pred = outputs.predictions.argmax(1)

labels = train_ds.features['label'].names
cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels)
fig = plt.figure(figsize=(20,20))
ax = fig.add_subplot()
disp.plot(xticks_rotation=90, ax = ax)
```

8 1C-1a: ViT Exercise – Age Classification with Custom ViTLayer

Keywords: ViT, custom ViTLayer, ViTAttention, ViTIntermediate, ViTOutput, residual connection, LayerNorm, mock.patch, Age Classification

8.1 Dataset Loading

```python
import numpy as np
from datasets import load_dataset, Dataset

dataset = load_dataset('prithivMLmods/Age-Classification-Set', split='train')

seed = 42
rng = np.random.default_rng(seed)

indices = np.arange(len(dataset))
rng.shuffle(indices)

train_indices = indices[:13300]
val_indices = indices[13300 : 13300+2850]
test_indices = indices[13300+2850 : ]

train_ds = dataset.select(train_indices)
val_ds = dataset.select(val_indices)
test_ds = dataset.select(test_indices)

print(f"Train dataset size: {len(train_ds)}")
print(f"Test dataset size: {len(test_ds)}")
```

8.2 Custom ViTLayer (Residual Connections)

```python
import torch
import torch.nn as nn
from transformers.models.vit.modeling_vit import ViTAttention, ViTIntermediate, ViTOutput
from typing import Optional, Tuple, Union
from transformers.models.vit.configuration_vit import ViTConfig

class ViTLayer(nn.Module):
    """Custom ViT block compatible with the installed Transformers version."""

    def __init__(self, config: ViTConfig) -> None:
        super().__init__()
        self.chunk_size_feed_forward = config.chunk_size_feed_forward
        self.seq_len_dim = 1
        self.attention = ViTAttention(config)
        self.intermediate = ViTIntermediate(config)
        self.output = ViTOutput(config)
        self.layernorm_before = nn.LayerNorm(config.hidden_size, eps=config.layer_norm_eps)
        self.layernorm_after = nn.LayerNorm(config.hidden_size, eps=config.layer_norm_eps)

    def forward(self, hidden_states: torch.Tensor) -> torch.Tensor:
        hidden_states_norm = self.layernorm_before(hidden_states)
        attention_output = self.attention(hidden_states_norm)

        # first residual connection
        hidden_states = attention_output + hidden_states

        # in ViT, layernorm is also applied after self-attention
        layer_output = self.layernorm_after(hidden_states)
        layer_output = self.intermediate(layer_output)

        # second residual connection is done here
        layer_output = self.output(layer_output, hidden_states)

        return layer_output
```

8.3 Load Model with Custom Layer (mock.patch)

```python
from transformers import ViTForImageClassification
with (mock.patch("transformers.models.vit.modeling_vit.ViTLayer", new=ViTLayer),):
    model = ViTForImageClassification.from_pretrained('google/vit-base-patch16-224-in21k',
                            num_labels=5,
                            id2label=id2label,
```

```
                                label2id=label2id,
                                )
```

8.4 Training with HF Trainer

```
from transformers import TrainingArguments, Trainer

metric_name = "accuracy"

args = TrainingArguments(
    f"ViT_finetune",
    save_strategy="epoch",
    eval_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=100,
    per_device_eval_batch_size=100,
    num_train_epochs=1,
    weight_decay=0.01,
    load_best_model_at_end=True,
    metric_for_best_model=metric_name,
    logging_dir='logs',
    logging_strategy="steps",
    logging_steps=50,
    remove_unused_columns=False,
    report_to=[],
)
```

8.5 Evaluation with Confusion Matrix and Classification Report

```
from pathlib import Path
from torch.utils.data import DataLoader
from transformers import ViTForImageClassification
import torch
from sklearn.metrics import confusion_matrix, classification_report
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

device = "cuda" if torch.cuda.is_available() else "cpu"
model.to(device)
model.eval()

predictions = []
true_labels = []

test_dataloader = DataLoader(test_ds, batch_size=32, collate_fn=collate_fn)

with torch.no_grad():
    for batch in test_dataloader:
        pixel_values = batch["pixel_values"].to(device)
        labels = batch["labels"].to(device)
        outputs = model(pixel_values=pixel_values)
        logits = outputs.logits
        predicted_labels = torch.argmax(logits, dim=-1)
        predictions.extend(predicted_labels.cpu().numpy())
        true_labels.extend(labels.cpu().numpy())

class_names = [id2label[i] for i in range(len(id2label))]

cm = confusion_matrix(true_labels, predictions)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=class_names, yticklabels=class_names)
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix")
plt.show()

cr = classification_report(true_labels, predictions, target_names=class_names)
print("\nClassification Report:")
print(cr)
```

9 1C-2: Swin Transformer Fine-Tuning (Butterfly Classification)

Keywords: Swin Transformer, swin_t, timm, LabelSmoothingCrossEntropy, ImageFolder, AdamW, StepLR, freeze, fine-tune, train_model

9.1 Imports

```
import numpy as np
import pandas as pd
import os
import torch
import torchvision
from torchvision import datasets
from torchvision import transforms as T
from torch import nn, optim
from torch.nn import functional as F
from torch.utils.data import DataLoader, sampler, random_split
from torchvision import models
import timm
from timm.loss import LabelSmoothingCrossEntropy
import warnings
warnings.filterwarnings("ignore")
import matplotlib.pyplot as plt
import sys
from tqdm import tqdm
import time
import copy
```

9.2 Data Loading Functions

```
def get_classes(data_dir):
    all_data = datasets.ImageFolder(data_dir)
    return all_data.classes

def get_data_loaders(data_dir, batch_size, train = False):
    if train:
        transform = T.Compose([
            T.RandomHorizontalFlip(),
            T.RandomVerticalFlip(),
            T.RandomApply(torch.nn.ModuleList([T.ColorJitter()]), p=0.25),
            T.Resize(256),
            T.CenterCrop(224),
            T.ToTensor(),
            T.Normalize(timm.data.IMAGENET_DEFAULT_MEAN, timm.data.IMAGENET_DEFAULT_STD),
            T.RandomErasing(p=0.1, value='random')
        ])
        train_data = datasets.ImageFolder(os.path.join(data_dir, "train/"), transform = transform)
        train_loader = DataLoader(train_data, batch_size=batch_size, shuffle=True, num_workers=4)
        return train_loader, len(train_data)
    else:
        transform = T.Compose([
            T.Resize(256),
            T.CenterCrop(224),
            T.ToTensor(),
            T.Normalize(timm.data.IMAGENET_DEFAULT_MEAN, timm.data.IMAGENET_DEFAULT_STD),
        ])
        val_data = datasets.ImageFolder(os.path.join(data_dir, "valid/"), transform=transform)
        test_data = datasets.ImageFolder(os.path.join(data_dir, "test/"), transform=transform)
        val_loader = DataLoader(val_data, batch_size=batch_size, shuffle=True, num_workers=4)
        test_loader = DataLoader(test_data, batch_size=batch_size, shuffle=True, num_workers=4)
        return val_loader, test_loader, len(val_data), len(test_data)
```

9.3 Model Setup (Freeze + Custom Head)

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

model = torchvision.models.swin_t(weights='IMAGENET1K_V1')

for param in model.parameters():
    param.requires_grad = False

n_inputs = model.head.in_features
model.head = nn.Sequential(
```

```
    nn.Linear(n_inputs, 512),
    nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(512, len(classes))
)
model = model.to(device)
print(model.head)

criterion = LabelSmoothingCrossEntropy()
criterion = criterion.to(device)
optimizer = optim.AdamW(model.head.parameters(), lr=0.001)

exp_lr_scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=3, gamma=0.97)
```

9.4 Training Function

```
def train_model(model, criterion, optimizer, scheduler, num_epochs=10):
    since = time.time()
    best_model_wts = copy.deepcopy(model.state_dict())
    best_acc = 0.0

    for epoch in range(num_epochs):
        print(f'Epoch {epoch}/{num_epochs - 1}')
        print("-"*10)

        for phase in ['train', 'val']:
            if phase == 'train':
                model.train()
            else:
                model.eval()

            running_loss = 0.0
            running_corrects = 0.0

            for inputs, labels in tqdm(dataloaders[phase]):
                inputs = inputs.to(device)
                labels = labels.to(device)

                optimizer.zero_grad()

                with torch.set_grad_enabled(phase == 'train'):
                    outputs = model(inputs)
                    _, preds = torch.max(outputs, 1)
                    loss = criterion(outputs, labels)

                    if phase == 'train':
                        loss.backward()
                        optimizer.step()
                running_loss += loss.item() * inputs.size(0)
                running_corrects += torch.sum(preds == labels.data)

            if phase == 'train':
                scheduler.step()

            epoch_loss = running_loss / dataset_sizes[phase]
            epoch_acc =  running_corrects.double() / dataset_sizes[phase]

            print("{} Loss: {:.4f} Acc: {:.4f}".format(phase, epoch_loss, epoch_acc))

            if phase == 'val' and epoch_acc > best_acc:
                best_acc = epoch_acc
                best_model_wts = copy.deepcopy(model.state_dict())
        print()
    time_elapsed = time.time() - since
    print('Training complete in {:.0f}m {:.0f}s'.format(time_elapsed // 60, time_elapsed % 60))
    print("Best Val Acc: {:.4f}".format(best_acc))

    model.load_state_dict(best_model_wts)
    return model
```

9.5 Testing

```
test_loss = 0.0
class_correct = list(0 for i in range(len(classes)))
class_total = list(0 for i in range(len(classes)))
```

```
model_ft.eval()

for data, target in tqdm(test_loader):
    data, target = data.to(device), target.to(device)
    with torch.no_grad():
        output = model_ft(data)
        loss = criterion(output, target)
    test_loss = loss.item() * data.size(0)
    _, pred = torch.max(output, 1)
    correct_tensor = pred.eq(target.data.view_as(pred))
    correct = np.squeeze(correct_tensor.cpu().numpy())
    if len(target) == 32:
        for i in range(32):
            label = target.data[i]
            class_correct[label] += correct[i].item()
            class_total[label] += 1

test_loss = test_loss / test_data_len
print('Test Loss: {:.4f}'.format(test_loss))
for i in range(len(classes)):
    if class_total[i] > 0:
        print("Test Accuracy of %5s: %2d%% (%2d/%2d)" % (
            classes[i], 100*class_correct[i]/class_total[i], np.sum(class_correct[i]), np.sum(class_total[i])
        ))
```

10 1D-1: Detection Transformer (DETR) Fine-Tuning

Keywords: DETR, DetrForObjectDetection, DetrImageProcessor, object detection, COCO, PyTorch Lightning, collate_fn, bounding box, post_process, CocoEvaluator

10.1 COCO Dataset Class

```
import torchvision
import os

class CocoDetection(torchvision.datasets.CocoDetection):
    def __init__(self, img_folder, feature_extractor, train=True):
        ann_file = os.path.join(img_folder, "_annotations.coco.json")
        super(CocoDetection, self).__init__(img_folder, ann_file)
        self.feature_extractor = feature_extractor

        cat_ids = sorted(self.coco.cats.keys())
        self.cat_id_to_contiguous = {cat_id: i for i, cat_id in enumerate(cat_ids)}
        self.contiguous_to_cat_id = {i: cat_id for cat_id, i in self.cat_id_to_contiguous.items()}

    def __getitem__(self, idx):
        img, annotations = super(CocoDetection, self).__getitem__(idx)

        image_id = self.ids[idx]
        remapped_annotations = []
        for ann in annotations:
            ann = ann.copy()
            ann["category_id"] = self.cat_id_to_contiguous[ann["category_id"]]
            remapped_annotations.append(ann)

        target = {"image_id": image_id, "annotations": remapped_annotations}
        encoding = self.feature_extractor(images=img, annotations=target, return_tensors="pt")
        pixel_values = encoding["pixel_values"].squeeze()
        target = encoding["labels"][0]

        return pixel_values, target
```

10.2 Feature Extractor and Datasets

```
from transformers import DetrImageProcessor

feature_extractor = DetrImageProcessor.from_pretrained("facebook/detr-resnet-101")

train_dataset = CocoDetection(img_folder=str(data_root / 'train'), feature_extractor=
    feature_extractor)
val_dataset = CocoDetection(img_folder=str(data_root / 'valid'), feature_extractor=feature_extractor
    , train=False)
```

10.3 DataLoader with Padding Collate

```
from torch.utils.data import DataLoader
import torch

def collate_fn(batch):
  pixel_values = [item[0] for item in batch]
  labels = [item[1] for item in batch]

  max_h = max(pv.shape[1] for pv in pixel_values)
  max_w = max(pv.shape[2] for pv in pixel_values)
  bsz = len(pixel_values)
  channels = pixel_values[0].shape[0]

  padded_pixel_values = torch.zeros((bsz, channels, max_h, max_w), dtype=pixel_values[0].dtype)
  pixel_mask = torch.zeros((bsz, max_h, max_w), dtype=torch.long)

  for i, pv in enumerate(pixel_values):
    h, w = pv.shape[1], pv.shape[2]
    padded_pixel_values[i, :, :h, :w] = pv
    pixel_mask[i, :h, :w] = 1

  batch = {}
  batch['pixel_values'] = padded_pixel_values
  batch['pixel_mask'] = pixel_mask
```

```
  batch['labels'] = labels
  return batch

train_dataloader = DataLoader(train_dataset, collate_fn=collate_fn, batch_size=2, shuffle=True)
val_dataloader = DataLoader(val_dataset, collate_fn=collate_fn, batch_size=2)
```

10.4 DETR Lightning Module

```
import pytorch_lightning as pl
from transformers import DetrConfig, DetrForObjectDetection
import torch

class Detr(pl.LightningModule):

     def __init__(self, lr, lr_backbone, weight_decay):
         super().__init__()
         self.model = DetrForObjectDetection.from_pretrained(
             "facebook/detr-resnet-101",
             num_labels=len(id2label),
             id2label=id2label,
             label2id=label2id,
             ignore_mismatched_sizes=True
         )
         self.lr = lr
         self.lr_backbone = lr_backbone
         self.weight_decay = weight_decay

     def forward(self, pixel_values, pixel_mask):
       outputs = self.model(pixel_values=pixel_values, pixel_mask=pixel_mask)
       return outputs

     def common_step(self, batch, batch_idx):
       pixel_values = batch["pixel_values"]
       pixel_mask = batch["pixel_mask"]
       labels = [{k: v.to(self.device) for k, v in t.items()} for t in batch["labels"]]

       outputs = self.model(pixel_values=pixel_values, pixel_mask=pixel_mask, labels=labels)

       loss = outputs.loss
       loss_dict = outputs.loss_dict

       return loss, loss_dict

     def training_step(self, batch, batch_idx):
        loss, loss_dict = self.common_step(batch, batch_idx)
        self.log("training_loss", loss)
        for k,v in loss_dict.items():
          self.log("train_" + k, v.item())
        return loss

     def validation_step(self, batch, batch_idx):
        loss, loss_dict = self.common_step(batch, batch_idx)
        self.log("validation_loss", loss)
        for k,v in loss_dict.items():
          self.log("validation_" + k, v.item())
        return loss

     def configure_optimizers(self):
        param_dicts = [
              {"params": [p for n, p in self.named_parameters() if "backbone" not in n and p.requires_grad]},
              {
                  "params": [p for n, p in self.named_parameters() if "backbone" in n and p.requires_grad],
                  "lr": self.lr_backbone,
              },
        ]
        optimizer = torch.optim.AdamW(param_dicts, lr=self.lr,
                                      weight_decay=self.weight_decay)
        return optimizer

     def train_dataloader(self):
        return train_dataloader

     def val_dataloader(self):
        return val_dataloader
```

10.5 Train and Evaluate

```
model = Detr(lr=1e-4, lr_backbone=5e-6, weight_decay=1e-4)

from pytorch_lightning import Trainer

trainer = Trainer(devices=1, max_steps=300, gradient_clip_val=0.1, log_every_n_steps=10)
trainer.fit(model)
```

10.6 COCO Evaluation

```
from datasets import get_coco_api_from_dataset
from datasets.coco_eval import CocoEvaluator
from tqdm.auto import tqdm

base_ds = get_coco_api_from_dataset(val_dataset)

iou_types = ['bbox']
coco_evaluator = CocoEvaluator(base_ds, iou_types)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
model.eval()

for idx, batch in enumerate(tqdm(val_dataloader)):
    pixel_values = batch["pixel_values"].to(device)
    pixel_mask = batch["pixel_mask"].to(device)
    labels = [{k: v.to(device) for k, v in t.items()} for t in batch["labels"]]

    outputs = model.model(pixel_values=pixel_values, pixel_mask=pixel_mask)

    orig_target_sizes = torch.stack([target["orig_size"] for target in labels], dim=0)
    results = feature_extractor.post_process_object_detection(outputs, threshold=0.8, target_sizes=
    orig_target_sizes)
    res = {target['image_id'].item(): output for target, output in zip(labels, results)}
    coco_evaluator.update(res)

coco_evaluator.synchronize_between_processes()
coco_evaluator.accumulate()
coco_evaluator.summarize()
```

10.7 Visualization Helpers

```
import torch
import matplotlib.pyplot as plt

COLORS = [[0.000, 0.447, 0.741], [0.850, 0.325, 0.098], [0.929, 0.694, 0.125],
          [0.494, 0.184, 0.556], [0.466, 0.674, 0.188], [0.301, 0.745, 0.933]]

def box_cxcywh_to_xyxy(x):
    x_c, y_c, w, h = x.unbind(1)
    b = [(x_c - 0.5 * w), (y_c - 0.5 * h),
         (x_c + 0.5 * w), (y_c + 0.5 * h)]
    return torch.stack(b, dim=1)

def rescale_bboxes(out_bbox, size):
    img_w, img_h = size
    b = box_cxcywh_to_xyxy(out_bbox)
    b = b * torch.tensor([img_w, img_h, img_w, img_h], dtype=torch.float32)
    return b

def plot_results(pil_img, prob, boxes):
    plt.figure(figsize=(16,10))
    plt.imshow(pil_img)
    ax = plt.gca()
    colors = COLORS * 100
    for p, (xmin, ymin, xmax, ymax), c in zip(prob, boxes.tolist(), colors):
        ax.add_patch(plt.Rectangle((xmin, ymin), xmax - xmin, ymax - ymin,
                                   fill=False, color=c, linewidth=3))
        cl = p.argmax()
        text = f'{id2label[cl.item()]}: {p[cl]:0.2f}'
        ax.text(xmin, ymin, text, fontsize=15,
                bbox=dict(facecolor='yellow', alpha=0.5))
    plt.axis('off')
    plt.show()
```

```python

def visualize_predictions(image, outputs, threshold=0.8):
  probas = outputs.logits.softmax(-1)[0, :, :-1]
  keep = probas.max(-1).values > threshold
  bboxes_scaled = rescale_bboxes(outputs.pred_boxes[0, keep].cpu(), image.size)
  plot_results(image, probas[keep], bboxes_scaled)
```

11 1D-2: DAB-DETR Inference

Keywords: DAB-DETR, AutoModelForObjectDetection, AutoImageProcessor, post_process_object_detection, COCO, bounding box visualization

11.1 Load Model

```
import torch
import requests

from PIL import Image
from transformers import AutoModelForObjectDetection, AutoImageProcessor

image_processor = AutoImageProcessor.from_pretrained("IDEA-Research/dab-detr-resnet-50")
model = AutoModelForObjectDetection.from_pretrained("IDEA-Research/dab-detr-resnet-50")
```

11.2 Fetch COCO Images

```
candidate_image_ids = [
    39769, 139, 285, 632, 724, 776, 785, 802, 872, 885,
    913, 939, 962, 974, 1000, 1059, 1072, 1122, 1132, 1152,
]

def fetch_coco_image(image_id):
    filename = f"{int(image_id):012d}.jpg"
    url = f"http://images.cocodataset.org/val2017/{filename}"
    try:
        response = requests.get(url, stream=True, timeout=12)
        response.raise_for_status()
        return filename, Image.open(response.raw).convert("RGB")
    except Exception:
        return None

selected_images = []
for image_id in candidate_image_ids:
    result = fetch_coco_image(image_id)
    if result is not None:
        selected_images.append(result)
    if len(selected_images) == 10:
        break
```

11.3 Detection and Visualization

```
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from collections import Counter

fig, axes = plt.subplots(2, 5, figsize=(25, 10))
axes = axes.flatten()

threshold = 0.3

all_detected_labels = []
per_image_labels = {}
for ax, (filename, image) in zip(axes, selected_images):
    inputs = image_processor(images=image, return_tensors="pt")
    with torch.no_grad():
        outputs = model(**inputs)

    results = image_processor.post_process_object_detection(
        outputs,
        target_sizes=torch.tensor([image.size[::-1]]),
        threshold=threshold
    )[0]

    ax.imshow(image)
    ax.set_title(filename, fontsize=10)
    ax.axis("off")

    image_labels = []
    for score, label_id, box in zip(results["scores"], results["labels"], results["boxes"]):
        x1, y1, x2, y2 = box.tolist()
        width, height = x2 - x1, y2 - y1
```

```python

        rect = patches.Rectangle(
            (x1, y1), width, height,
            linewidth=2, edgecolor="lime", facecolor="none"
        )
        ax.add_patch(rect)

        label = model.config.id2label[label_id.item()]
        image_labels.append(label)
        all_detected_labels.append(label)

        ax.text(
            x1, max(0, y1 - 5),
            f"{label}: {score.item():.2f}",
            color="white", fontsize=8,
            bbox=dict(facecolor="black", alpha=0.6, pad=2)
        )

    per_image_labels[filename] = sorted(set(image_labels))

plt.tight_layout()
plt.show()
```

11.4 Statistics

```python
label_counter = Counter(all_detected_labels)
print(f"threshold = {threshold}")
print(f"Total Objects = {len(all_detected_labels)}")
print(f"Number of Detected Object Categories = {len(label_counter)}")
print("\nNumber of Objects per Category:")
for label, count in label_counter.most_common():
    print(f"- {label}: {count}")

print("\nDetected Object Categories per Image:")
for filename, labels in per_image_labels.items():
    label_text = ", ".join(labels) if labels else "(none)"
    print(f"- {filename}: {label_text}")
```

12 1E-1: ViT Self-Attention Exercise (DINO Attention Visualization)

Keywords: ViT, self-attention, DINO, attention overlay, heatmap, query key value, transpose_for_scores, scaled dot product

12.1 ViTSelfAttentionExercise Class

```python
import math
import torch
import torch.nn as nn

class ViTSelfAttentionExercise(nn.Module):
    def __init__(self, hidden_size: int = 384, num_attention_heads: int = 6, dropout_prob: float = 0.0):
        super().__init__()
        if hidden_size % num_attention_heads != 0:
            raise ValueError("hidden_size must be divisible by num_attention_heads")

        self.hidden_size = hidden_size
        self.num_attention_heads = num_attention_heads
        self.attention_head_size = hidden_size // num_attention_heads
        self.all_head_size = self.num_attention_heads * self.attention_head_size

        self.query = nn.Linear(hidden_size, self.all_head_size)
        self.key = nn.Linear(hidden_size, self.all_head_size)
        self.value = nn.Linear(hidden_size, self.all_head_size)
        self.dropout = nn.Dropout(dropout_prob)

    def transpose_for_scores(self, x: torch.Tensor) -> torch.Tensor:
        new_shape = x.size()[:-1] + (self.num_attention_heads, self.attention_head_size)
        x = x.view(new_shape)
        return x.permute(0, 2, 1, 3)

    def forward(self, hidden_states: torch.Tensor, output_attentions: bool = False):
        # TODO: replace '_' with query/key/value
        '_' = self.transpose_for_scores(self.query(hidden_states))
        '_' = self.transpose_for_scores(self.key(hidden_states))
        '_' = self.transpose_for_scores(self.value(hidden_states))

        attention_scores = torch.matmul(query, key.transpose(-1, -2))
        attention_scores = attention_scores / math.'FUNCTION'(self.attention_head_size)
        # TODO: replace 'FUNCTION' with sqrt

        attention_probs = torch.'ACTIVATION'(attention_scores, dim=-1)
        # TODO: replace 'ACTIVATION' with softmax

        context_layer = torch.matmul(attention_probs, value)
        context_layer = context_layer.permute(0, 2, 1, 3).contiguous()
        new_context_shape = context_layer.size()[:-2] + (self.all_head_size,)
        context_layer = context_layer.view(new_context_shape)

        if output_attentions:
            return context_layer, attention_probs
        return context_layer,
```

12.2 Test Script (Weight Copy and Verification)

```python
import torch
from transformers import ViTModel
from vit_self_attention_exercise import ViTSelfAttentionExercise

def copy_weights_from_hf(hf_attn, exercise_attn):
    exercise_attn.query.weight.data.copy_(hf_attn.query.weight.data)
    exercise_attn.query.bias.data.copy_(hf_attn.query.bias.data)
    exercise_attn.key.weight.data.copy_(hf_attn.key.weight.data)
    exercise_attn.key.bias.data.copy_(hf_attn.key.bias.data)
    exercise_attn.value.weight.data.copy_(hf_attn.value.weight.data)
    exercise_attn.value.bias.data.copy_(hf_attn.value.bias.data)

def main():
    model = ViTModel.from_pretrained("facebook/dino-vitb8", add_pooling_layer=False)
    model.eval()

    hf_attn = model.encoder.layer[0].attention.attention
    exercise_attn = ViTSelfAttentionExercise(
```

```
        hidden_size=model.config.hidden_size,
        num_attention_heads=model.config.num_attention_heads,
        dropout_prob=model.config.attention_probs_dropout_prob,
    )
    exercise_attn.eval()
    copy_weights_from_hf(hf_attn, exercise_attn)

    x = torch.randn(2, 197, model.config.hidden_size)

    with torch.no_grad():
        hf_context = hf_attn(x, output_attentions=False)[0]
        ex_context = exercise_attn(x, output_attentions=False)[0]

    max_abs_diff = (hf_context - ex_context).abs().max().item()
    print(f"max_abs_diff={max_abs_diff:.8f}")
    if max_abs_diff < 1e-5:
        print("PASS")
    else:
        print("NOT MATCHED")
```

12.3 DINO Attention Visualization (Notebook)

```
import argparse
import os
import numpy as np
import requests
import torch
from PIL import Image
from transformers import ViTImageProcessor, ViTModel, ViTConfig
from vit_self_attention_exercise import ViTSelfAttentionExercise

def copy_weights_from_hf(hf_attn, exercise_attn):
    exercise_attn.query.weight.data.copy_(hf_attn.query.weight.data)
    exercise_attn.query.bias.data.copy_(hf_attn.query.bias.data)
    exercise_attn.key.weight.data.copy_(hf_attn.key.weight.data)
    exercise_attn.key.bias.data.copy_(hf_attn.key.bias.data)
    exercise_attn.value.weight.data.copy_(hf_attn.value.weight.data)
    exercise_attn.value.bias.data.copy_(hf_attn.value.bias.data)

def load_image(image_path_or_url: str) -> Image.Image:
    if image_path_or_url.startswith("http://") or image_path_or_url.startswith("https://"):
        return Image.open(requests.get(image_path_or_url, stream=True).raw).convert("RGB")
    return Image.open(image_path_or_url).convert("RGB")

def normalize_map(attn_map: np.ndarray) -> np.ndarray:
    attn_map = attn_map.astype(np.float32)
    return (attn_map - attn_map.min()) / (attn_map.max() - attn_map.min() + 1e-8)

config = ViTConfig.from_pretrained("facebook/dino-vits8")
config.output_attentions = True
model = ViTModel.from_pretrained("facebook/dino-vits8", config=config, add_pooling_layer=False)
model.eval()

processor = ViTImageProcessor.from_pretrained("facebook/dino-vits8")

image = load_image("https://encrypted-tbn0.gstatic.com/images?q=...")
inputs = processor(images=image, return_tensors="pt")

with torch.no_grad():
    outputs = model(**inputs, output_attentions=True)
    attention_probs = outputs.attentions[-1]

cls_to_patches = attention_probs[0, :, 0, 1:]
mean_attn = cls_to_patches.mean(dim=0)
num_patches_side = int(mean_attn.numel() ** 0.5)
attn_map = mean_attn.view(num_patches_side, num_patches_side).cpu().numpy()
attn_map = normalize_map(attn_map)
```